Balatdle UML diagrams

These diagrams describe the first iteration of Balatdle. They follow the requirements in Requirements.md, and the class diagram follows the database design in design.md. The PlantUML source for each diagram sits next to its exported images in `uml-diagrams/`.

1. Use-case diagram

The system boundary is the rectangle labeled Balatdle, and everything inside it is behavior the software provides. The three actors sit outside the boundary because they are not part of the system.

Actors

Player is anyone playing the game. There is no account, so every player is anonymous and is recognized only by a token stored in their browser.

Administrator is me in practice, and this actor loads the joker roster and decides which joker is the answer on which date, neither of which a player can reach.

Clock is a time actor. UML allows an actor that is not a person when something outside the system triggers behavior, and the daily rollover at 00:00 UTC is set off by the time of day.

Use cases

Play Daily Puzzle is the main use case, where the player opens the game and works on today's joker until they name it correctly. Guesses are unlimited, so the round ends on a win rather than on running out of attempts. It has two extension points, "round won" and "during the round", which are where the two extending use cases attach.

Play Archived Puzzle runs the same loop against a date that has already passed, so a player can catch up on a day they missed, and the result does not affect their streak.

Submit Guess is one attempt, where the player names a joker and the system records it and returns feedback.

Validate Joker Name is the check the server runs against the roster before anything else, and a name that is not a real joker is rejected without counting as an attempt.

Evaluate Guess compares the guessed joker to the answer across all six attributes and builds the feedback. This happens on the server, so the answer never reaches the browser before the puzzle is solved.

Identify Returning Player looks up the token from the browser and finds the matching player record, or creates one on a first visit, which is what lets streaks and progress survive without an account.

View Statistics shows the player their games played, win percentage, current streak, and longest streak.

Share Result copies a spoiler-free grid of the player's result so they can paste it somewhere else.

View Rules opens the panel explaining the six attributes and what each feedback color means.

Seed Joker Roster is the administrator loading the joker data from `jokers.json` into the database, and nothing else works until this has been done once.

Schedule Daily Puzzle is the administrator assigning jokers to future dates, so the answer for a given day is decided in advance rather than picked at midnight.

Advance to Next Puzzle closes the current puzzle at 00:00 UTC and makes the next date active.

Relationships

An `<<include>>` means the base use case always performs the included one, with no condition attached.

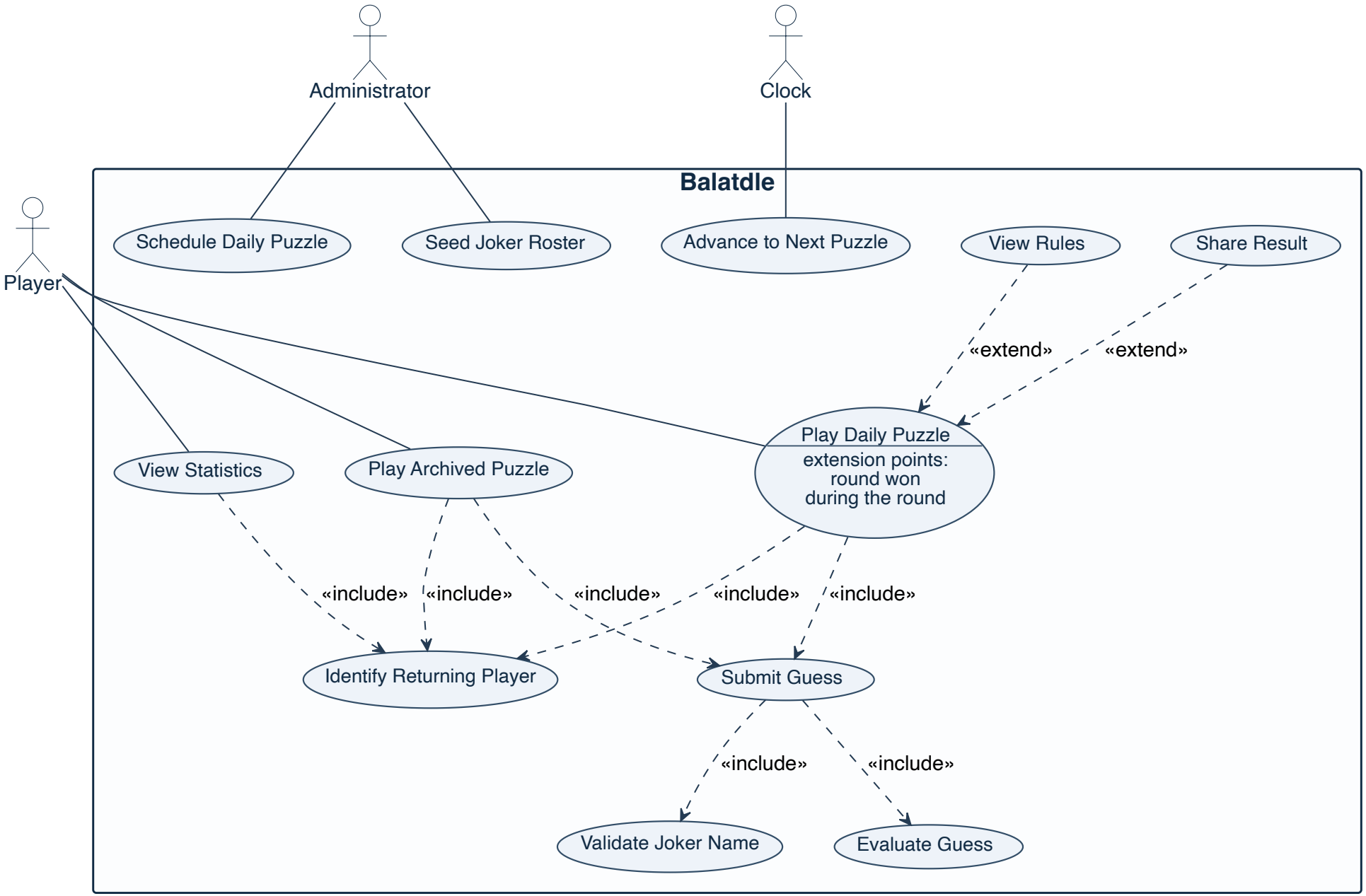
- Play Daily Puzzle includes Submit Guess, since guessing is the whole of a round.
- Play Archived Puzzle includes Submit Guess, because both ways of playing run the same guessing behavior, and factoring it out is the reason to use an include here instead of drawing the same steps twice.
- Submit Guess includes Validate Joker Name, so every guess is checked against the roster with no path that skips it.
- Submit Guess includes Evaluate Guess, since the feedback is the point of making a guess.
- Play Daily Puzzle, Play Archived Puzzle, and View Statistics each include Identify Returning Player, because all three read or write records that belong to one player, and the system has to know whose records they are first.

An `<<extend>>` means the extending use case adds behavior to the base only under some condition, and the base is complete without it.

- Share Result extends Play Daily Puzzle at the "round won" extension point, because the share grid only exists once the puzzle is solved, and a player who has not won has nothing to share, which is why this cannot be an include.
- View Rules extends Play Daily Puzzle at the "during the round" extension point, since a player can open the rules at any point or never open them at all and the round still works.

The two extending use cases have no line of their own to the Player actor, because the player reaches them through Play Daily Puzzle, the use case they extend.

Balatdle: use cases



2. Sequence diagram

This diagram covers Submit Guess, the use case that generalizes the whole application, since every other feature is either setup for this flow or a view of what it produced.

Participants

The Player is the actor, and ReactUI is the frontend running in the browser. FlaskAPI is the backend, the same FlaskAPI class that appears in the class diagram, and the request reaches its submitGuess method through `POST /api/guess`. GuessEvaluator holds the comparison logic, Repository is the data access layer, and PostgreSQL is the database itself, drawn with the database symbol.

Message types

Solid arrows with filled heads are synchronous calls where the caller waits for an answer, and dashed arrows are return messages. The arrow from ReactUI back to itself is a self-call, used when the frontend filters the roster locally to build the autocomplete list without going to the server, and GuessEvaluator also calls itself once per attribute. The vertical bars on the lifelines are activation bars, showing when each object is doing work. Every call between backend objects matches an operation in the class diagram, with the same parameters.

Interaction fragments

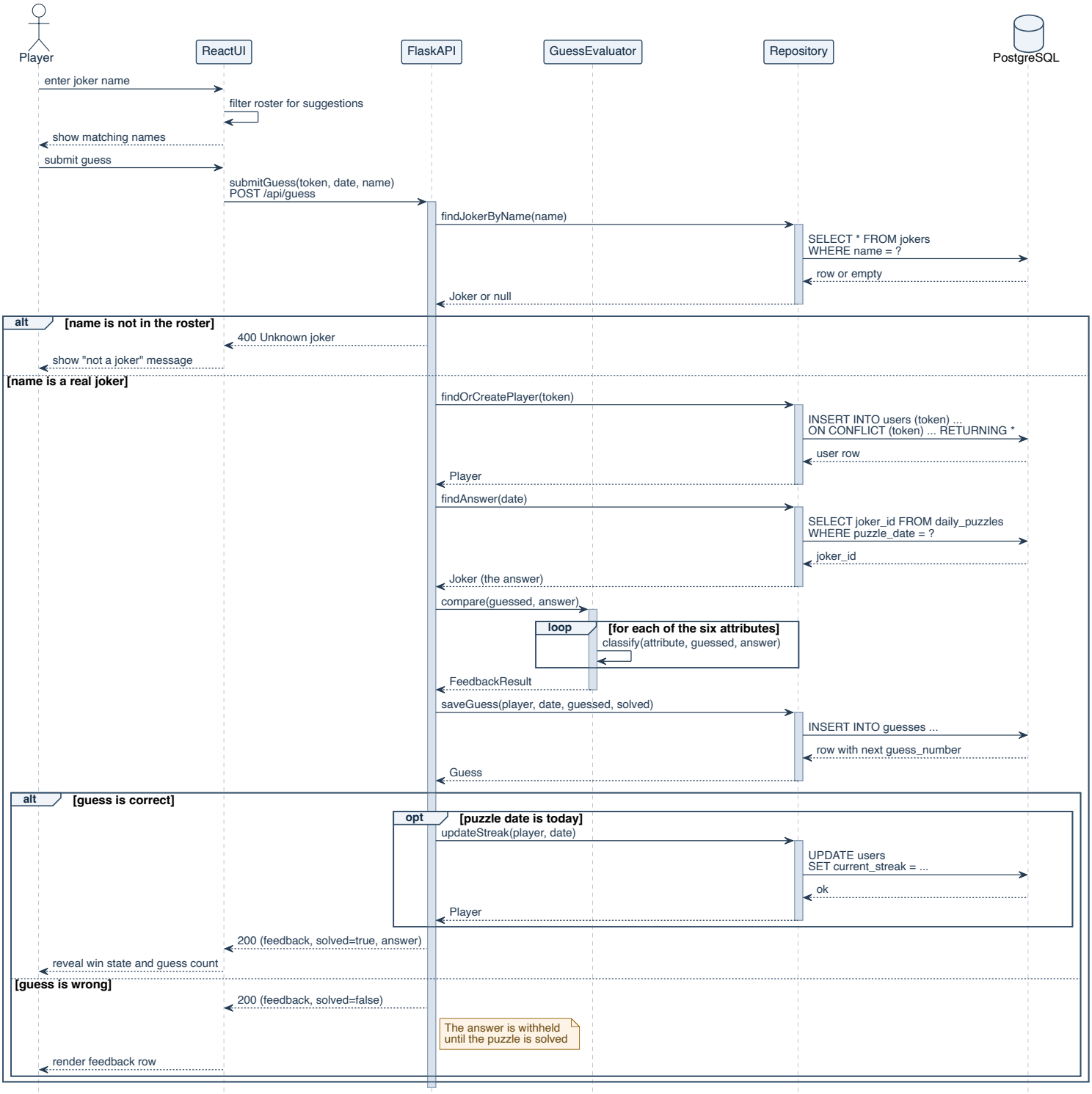
The outer `alt` splits on whether the submitted name is a real joker, and if it is not, the API returns a 400 and the flow ends there, which is the point of validating first.

In the valid branch the API first looks up the player from the browser token, creating the record on a first visit, which is the Identify Returning Player use case from the first diagram. A `loop` then runs the attribute comparison six times, once per compared attribute, and that is what produces the green, yellow, and arrow results.

A second `alt` splits on whether the guess was correct. On a win the response includes the answer, while on a wrong guess it carries feedback only, and the note on that branch marks the difference because sending the answer early would break NFR-1.

Inside the win branch, an `opt` fragment updates the streak only when the puzzle date is today, so solving an archived puzzle leaves the streak alone, which matches the rule in the use-case section.

Balatdle: submit a guess and receive feedback



3. Class diagram

The class diagram covers the Python backend. The React frontend appears in the sequence diagram as ReactUI, but it is not part of this class model.

Entity classes

Joker, DailyPuzzle, Player, and Guess map to the four database tables in `schema.sql`. Their attributes match the columns with the types written as Python types, and each foreign key is drawn as an association line instead of being repeated as an attribute, so a Guess reaches its Player, DailyPuzzle, and Joker through the lines on the diagram.

Supporting classes

FlaskAPI is the backend's entry point, and its three public methods are the three endpoints from the project plan, which get the day's puzzle, check a guess, and return a player's statistics. Repository is the only class that touches the database, so every SQL statement lives in one place.

GuessEvaluator holds the comparison logic and nothing else, taking two jokers and returning a FeedbackResult. Its private classify method picks the rule for each attribute and hands off to classifyRarity, classifyCost, or classifyText depending on the kind of value, and keeping the class free of database access is what makes those partial-match rules testable on their own.

FeedbackResult is what a guess produces. It is made of six AttributeFeedback parts, one per compared attribute, each pairing an attribute's name with its MatchLevel, and it also carries the CostDirection for the higher or lower arrow and can render itself as the spoiler-free share grid.

MatchLevel and CostDirection are enumerations, and MatchLevel has exactly three values, which is what makes green, yellow, and gray the complete set of outcomes for an attribute.

Relationships and cardinality

Relationship	Cardinality	Meaning
Joker is the answer for DailyPuzzle	1 to 0..*	One joker can be the answer on many dates, and a date has exactly one answer.
Joker is guessed as Guess	1 to 0..*	A joker can be guessed any number of times, including never.
Player makes Guess	1 to 0..*	A new player has no guesses yet, which is why the lower bound is zero.
DailyPuzzle receives Guess	1 to 0..*	Every guess belongs to exactly one puzzle date.
Guess produces FeedbackResult	1 to 1	Each guess yields exactly one result.

Relationship	Cardinality	Meaning
FeedbackResult holds AttributeFeedback, one per attribute	1 to 6	Composition, with one part for each of the six compared attributes.
AttributeFeedback has a MatchLevel as its outcome	1 to 1	The result for that one attribute.
FeedbackResult has a CostDirection as its cost arrow	1 to 1	The arrow shown on the cost column.
GuessEvaluator creates FeedbackResult	1 to 1	A dependency, drawn dashed, because the evaluator builds the result but does not own it.
FlaskAPI uses GuessEvaluator	1 to 1	The API holds one evaluator and calls it for every guess.
FlaskAPI uses Repository	1 to 1	All database work goes through the one repository instance.
Repository loads Joker	1 to 0..*	The repository reads the whole roster into memory for name validation.

The filled diamond between FeedbackResult and AttributeFeedback is composition, since each part has no meaning outside the result that contains it and is discarded with it. MatchLevel and CostDirection connect through plain associations, because an enumeration value is shared by every result that uses it and is never owned by one.

Balatdle: class structure

